
Universidade Estadual do Piauí – UESPI
Campus de Parnaíba
Tecnologia em Sistemas de Computação

Introdução a Computação

Aula 08 – Armazenamento de Dados
2 - Números Inteiros

Prof. Francisco Rocha

1

Armazenamento de Dados

• Percurso

– Representação e Armazenamento de Dados

- 1 - Introdução
- **2 - Números Inteiros**
- 3 - Números Reais
- 4 - Texto
- 5 - Imagem, Áudio e Vídeo

2

Armazenamento de Dados

• Armazenamento de Números

- Um número é convertido para o sistema binário antes de ser armazenado na memória do computador.
- Existem duas questões no armazenamento de números.
 - Como armazenar o ponto decimal.
 - Como armazenar o sinal do número.

3

Armazenamento de Dados

• Armazenamento de Números

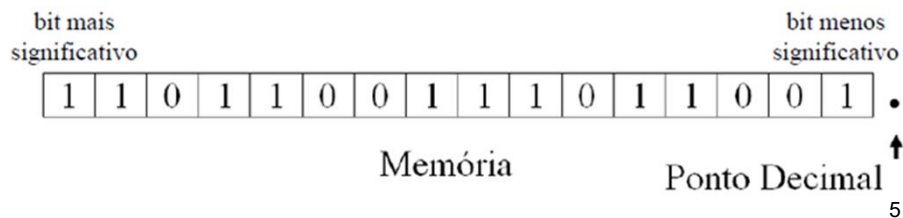
- Quanto ao armazenamento do ponto decimal:
 - Ponto fixo
 - Números inteiros: número sem uma parte fracionária.
 - Ex.: 134, 125, -321, -125
 - Ponto flutuante
 - Números reais: números com uma parte fracionária.
 - Ex.: 134.23, 0.125, -121.17 e -0.625

4

Armazenamento de Dados

• Armazenamento de Números Inteiros

- Nos inteiros, considera-se que o ponto decimal tem posição fixa.
 - Sempre à direita do bit menos significativo.
- Daí, chamada de **representação de ponto fixo**.
 - Ex.: 134 e 125, também representados como 134.0 e 125.0
 - Como o valor à direita da casa decimal seria zero, não precisamos representar no computador, pois não terá valor.



Armazenamento de Dados

• Números Inteiros Sem Sinal

- Podemos armazenar número inteiros com e sem sinal.
- Um número inteiro sem sinal nunca pode ser negativo.
 - Assume o valor 0 ou valores positivos.
- O intervalo dos números inteiros sem sinal vai de 0 ao infinito positivo.
- A maioria dos computadores define uma constante
 - **máximo número inteiro sem sinal**, que tem valor de $(2^n - 1)$
 - Onde n é o número de bits usado na palavra de computador.

Armazenamento de Dados

• Números Inteiros Sem Sinal

- Exemplo do maior inteiro representável de acordo com o número de bits:

Quantidade de Bits	Maior Inteiro Representável
1 bit	$2^1 - 1 = 1$
8 bits (1 byte)	$2^8 - 1 = 255$
16 bits (2 bytes)	$2^{16} - 1 = 65.535$
32 bits (4 bytes)	$2^{32} - 1 = 4.294.967.295$
64 bits (8 bytes)	$2^{64} - 1 = 18.446.744.073.709.551.615$

7

Armazenamento de Dados

• Números Inteiros Sem Sinal

- Podemos armazenar um número inteiro sem sinal utilizando as seguintes etapas:
 - Transforme o número inteiro para binário.
 - Se o número de bits for menor do que n.
 - 0s são adicionados à esquerda do número binário.
 - Se o número de bits for igual a n.
 - Armazene o número.
 - Se o número de bits for maior do que n.
 - O número inteiro não pode ser armazenado.

8

Armazenamento de Dados

• Números Inteiros Sem Sinal

– Exemplo: Armazene +7 em uma palavra de memória de 8 bits.

- Transforme o número inteiro em binário.

$(111)_2$

- Acrescente cinco 0s à esquerda para obter um total de 8 bits.

$(0000\ 0111)_2$

- Armazene o número na memória.

Armazenamento de Dados

• Números Inteiros Sem Sinal

– Exemplo: Armazene +258 em uma palavra de 16 bits.

- Transforme o número inteiro em binário.

$(100000010)_2$

- Acrescente sete 0s à esquerda para obter um total de 16 bits.

$(0000\ 0001\ 0000\ 0010)_2$

- Armazene o número na memória.

Armazenamento de Dados

- **Números Inteiros Sem Sinal**

- Exemplo: Armazene +255 em uma palavra de 8 bits.

- Transforme o número inteiro em binário,

$(11111111)_2$

- O número já ocupa os 8 bits.

$(1111\ 1111)_2$

- Armazene o número na memória.

11

Armazenamento de Dados

- **Números Inteiros Sem Sinal**

- Para recuperar um número da memória o padrão binário é convertido para decimal. Exemplo:

- Número na memória: $(0010\ 1011)_2$

- Qual o número convertido para decimal?

12

Armazenamento de Dados

• Números Inteiros Sem Sinal

- O intervalo de números inteiros que pode ser representado é limitado.
 - Devido a limitações de tamanho, isto é, o número de bits.
- Em uma posição de memória com n bits, podemos representar números entre 0 e $(2^n - 1)$, e.g. com 4 bits:

$$0 \text{ e } (2^4 - 1) = \text{números entre 0 e 15}$$

Vide a tabela do slide 7 para verificar exemplos dos maiores inteiros representáveis de acordo com o número de bits.

13

Armazenamento de Dados

• Números Inteiros Sem Sinal: overflow

- Quando tentamos armazenar um número maior do que o maior número do intervalo, ocorre o que chamamos de **Overflow**.
 - Ocorre um transbordamento.
 - Como exemplo, tente armazenar o número 20 em 4 bits, em seguida tente recuperá-lo para decimal.
- Overflow também ocorre em operações matemáticas ao tentar adicionar um valor maior do que se pode representar.
 - Como exemplo, tente armazenar o número inteiro 11 em um local de memória de 4 bits e tente adicionar 9 a esse número.

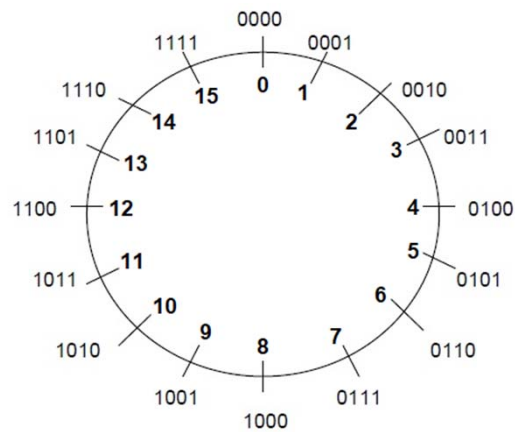
$$11_{10} + 9_{10} = 1011_2 + 1001_2 = (?)_2$$

14

Armazenamento de Dados

• Números Inteiros Sem Sinal

– Pegue o número 11 e some com 9. Que resultado obteve?



15

Armazenamento de Dados

• Números Inteiros Sem Sinal

– Como exemplo, qual é o resultado de $15 + 1$?

$$15_{10} + 1_{10} = (?)_{10}$$

– Considerando número binário com 4 bits, qual é o resultado de $15 + 1$?

$$1111_2 + 0001_2 = (?)_2$$

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

16

Armazenamento de Dados

- **Números Inteiros Sem Sinal**

– Qual é o resultado das seguintes operações:

- $11 + 00 = ?$
- $11 + 01 = ?$
- $0011 + 0111 = ?$
- $1111 + 0001 = ?$

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

17

Armazenamento de Dados

- **Números Inteiros Sem Sinal**

– No computador, os números sem sinal são usados para:

- Contagem
- Endereçamento
- Armazenamento de dados
 - Texto
 - Imagens
 - Áudio
 - Vídeo

18

Armazenamento de Dados

- **Números Inteiros Com Sinal**

- Para cada número na base 10 abaixo, diga se é positivo ou negativo.
 - +525
 - -525
 - 525

19

Armazenamento de Dados

- **Números Inteiros Com Sinal**

- Para cada número decimal abaixo, diga se é positivo ou negativo.
 - +525
 - -525
 - 525
- Utilizamos o símbolo de '-' para indicar quando um número é negativo e o de '+' para indicar quando é positivo.
 - Por convenção, o símbolo de '+' pode ser omitido.

20

Armazenamento de Dados

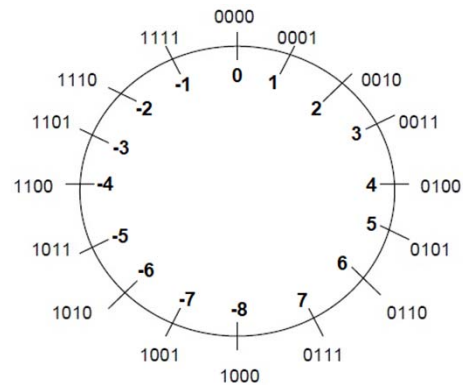
• Números Inteiros Com Sinal

– Para cada número binário em 4 bits abaixo, diga se é positivo ou negativo.

• 0101

• 1101

• Nos exemplos acima é necessário utilizar um bit para indicar se um número é negativo ou positivo.



21

Armazenamento de Dados

• Números Inteiros Com Sinal

– Percebe-se que o intervalo disponível para os números inteiros com sinal é dividido em dois subintervalos iguais, metade para positivos e metade para negativos.

– Veremos duas formas de representar números com sinal:

- sinal magnitude e
- complemento de dois.

– Uma das abordagens para representar **números inteiros com sinal** é conhecida como **sinal-magnitude**.

– Não é muito utilizada para inteiros, mas sim para armazenar parte de um número real.

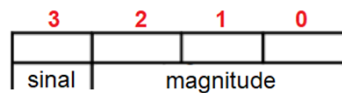
22

Armazenamento de Dados

• Inteiros Com Sinal: sinal magnitude

- Em sinal magnitude, o bit mais significativo é utilizado para armazenar o sinal.

- Como exemplo, números de 4 bits, utilizam apenas 3 bits para representar o valor absoluto do número.



- O intervalo é dividido em duas metades (ao lado).

- Para uma alocação de n bits é de

$$-(2^{n-1} - 1) \text{ até } +(2^{n-1} - 1)$$

$$-7 \text{ até } +7$$

7: 0111
6: 0110
5: 0101
4: 0100
3: 0011
2: 0010
1: 0001
0: 0000
-1: 1111
-2: 1110
-3: 1101
-4: 1100
-5: 1011
-6: 1010
-7: 1001
-8: 1000

23

Armazenamento de Dados

• Números Inteiros Com Sinal

- Exemplo de valores máximos de acordo com o tipo de dado:

Tipo	Memória consumida	Valor Mínimo	Valor Máximo
byte	1 byte	-128	127
short	2 byte	-32.768	32.767
int	4 bytes	-2.147.483.648	2.147.483.647
long	8 bytes	-9.223.372.036.854.775.808	9.223.372.036.854.775.807

24

Armazenamento de Dados

• Números Inteiros Com Sinal

- Regras para armazenamento de números com sinal na representação de sinal magnitude:
 - Transforme o número par para binário com n-1 bits;
 - Acrescenta o sinal;
 - Armazene o número.
- Como exemplos:
 - a) Armazene +28 em 8 bits.
 - b) Armazene -28 em 8 bits.
 - c) Recupere o número armazenado como 01001101.
 - d) Recupere o número armazenado como 10100001.

25

Armazenamento de Dados

• Números Inteiros Com Sinal

- Soluções:
 - a) 28 em sete bits: 0011100. Acrescenta sinal: 00011100 = +28.
 - b) 28 em sete bits: 0011100. Acrescenta sinal: 10011100 = -28.
 - c) Recupere o número armazenado como 01001101.
 - Bit mais à esquerda é 0, então o sinal é positivo.
 - O número inteiro é 77, sendo o valor recuperado: +77.
 - d) Recupere o número armazenado como 10100001.
 - Bit mais à esquerda é 1, então o sinal é negativo.
 - O número inteiro é 33, sendo o valor recuperado: -33.

26

Armazenamento de Dados

• Números Inteiros Com Sinal

- Overflow também ocorre na representação de sinal magnitude.
 - Nesse caso, temos overflow positivo e negativo.
- Exemplo de Overflow:
 - Considerando 4 bits para representar um número binário, some 6 ao número 5.

27

Armazenamento de Dados

• Números Inteiros Com Sinal

- Overflow de 4 bits na representação Sinal Magnitude:

0 : 0000	0101 (+5)	-0 : 1000
1 : 0001	<u>+ 0110</u> (+6)	-1 : 1001
2 : 0010	1011 (-3)	-2 : 1010
3 : 0011		-3 : 1011
4 : 0100	- 3	-4 : 1100
5 : 0101	↑ ↑	-5 : 1101
6 : 0110	1 011	-6 : 1110
7 : 0111		-7 : 1111

28

Armazenamento de Dados

• Números Inteiros Com Sinal

- A representação de sinal magnitude não é utilizada para representar números inteiros, mas sim para números reais.
- A representação utilizada em quase todos os computadores é a Complemento de Dois.
- Complemento de Dois
 - Armazena um número inteiro com sinal em n bits de memória.
 - O intervalo de números é dividido em duas partes.
 - Uma parte para positivos e outra para negativos.

29

Armazenamento de Dados

• Números Inteiros Com Sinal

- Em complemento de dois, numa representação de 4 bits, temos:
 - O intervalo é de 0000 a 1111
 - De 0000 até 0111 para positivos e
 - De 1000 até 1111 para negativos.
- O intervalo de valores é:

$-2^{(n-1)}$ a $2^{(n-1)} - 1$	0: 0000	-8: 1000
$-2^{(4-1)}$ a $2^{(4-1)} - 1$	1: 0001	-7: 1001
$-2^{(3)} a 2^{(3)} - 1$	2: 0010	-6: 1010
	3: 0011	-5: 1011
	4: 0100	-4: 1100
	5: 0101	-3: 1101
	6: 0110	-2: 1110
	7: 0111	-1: 1111

30

Armazenamento de Dados

• Números Inteiros Com Sinal

- Para converter um número binário em complemento dois:
 - Inverter todos os bits que representam o número binário; e
 - Some 1 aos bits invertidos.

- **Exemplo 1:** Considere o número $(-5)_{10}$. Qual seu valor binário em complemento de dois, em uma representação de 4 bits.

$$\begin{array}{r} 0101 (+5) \\ 0000 \\ 1010 \\ + \quad 1 \\ \hline 1011 (-5) \end{array}$$

- **Exemplo 2:** Considere o número $(-3)_{10}$. Qual seu valor binário em complemento de dois, em uma representação de 4 bits.

31

Armazenamento de Dados

• Números Inteiros Com Sinal

- Para converter um número binário em complemento de dois para decimal:
 - Se o número for positivo, converter o número para decimal.
 - Se o número for negativo, converter para complemento de dois, em seguida, converter para decimal.
 - Adicionar o sinal de negativo.

- Converta para decimal os valores binários abaixo armazenados em 4 bits com a representação complemento de dois:

a) 0110

b) 1110

$$\begin{array}{r} 0001 \\ 0001 \\ + \quad 1 \\ \hline 0010 (+2) \end{array}$$

$$\begin{array}{l} \text{Assim,} \\ 1110 = (-2) \end{array}$$

32

Armazenamento de Dados

• Números Inteiros Com Sinal

- **Desafio 1:** Utilize 4 bits para representar o números decimais a seguir em complemento de dois.

a) 4	c) 5	e) 12	g) 14
b) -4	d) -5	f) -12	h) -14

- **Desafio 2:** Utilize 8 bits para representar o números decimais a seguir em complemento de dois.

a) 15	c) 32	e) 46	g) 78
b) -15	d) -32	f) -46	h) -78

Armazenamento de Dados

• Referências Bibliográficas

- Behrouz Forouzan e Firouz Mosharraf. Fundamentos da Ciência da Computação. 2ed. Cengage Learning, 2012.
- J. Gleen Brookshear. Ciência da Computação: uma visão abrangente. 11ed. Bookman, 2013.
- Fernando C. Veloso. Informática: conceitos básicos. 8.ed. Elsevier, 2011.
- Marcelo Marcula; Pio Armando Benini Filho. Informática: conceitos e aplicações. 4ed. Érica, 2013.
- Ricardo Daniel Fedeli; Enrico Giulio Franco Polloni; Fernando Eduardo Peres. Introdução à ciência da computação. 2ed. Cengage Learning, 2010.

Armazenamento de Dados

